

Reviewer Pack — Coxeter Group Enumeration Complexity of Boolean Function Ent...

Entry 7a849ca4e68c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 04:05:46 UTC

Coxeter Group Enumeration Complexity of Boolean Function Entropy via Reflection Posets

Entry ID: 7a849ca4e68c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 04:05:46 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Group Theory (specifically, Coxeter groups and reflection posets) **Field B** (complexity object): Boolean function entropy

Statement:

For every d -regular graph G with n vertices, the minimal number of reflections in a Coxeter group that generate a subgroup isomorphic to the automorphism group of G is linearly correlated with its Shannon entropy $h(G)$, such that $|R(G)| = \Theta(h(G))$.

Rationale (proposer's reasoning):

Reflection posets encode the structure of Coxeter groups and have been used to study various combinatorial problems. A direct mapping between these structures and the entropy of boolean functions could reveal new insights into both areas by exposing a hidden connection.

Taxonomy category: COXETER_GROUP_ENTROPY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 26f9d63ce1447d1e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated d -regular graphs G with n vertices ($n \leq 40$), the correlation coefficient r of $|R(G)|$ and $h(G)$ over 30 random seeds is greater than or equal to 0.7, and there is no seed where $|R(G)| > 4h(G)$. The conjecture is falsified if either the correlation coefficient r is less than 0.7 for any seed or any seed produces a value of $|R(G)|$ that exceeds $4h(G)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter groups" AND "reflection posets" AND "Boolean function entropy" - "automorphism group" AND "d-regular graph" AND "Shannon entropy" - "minimal number of reflections" IN COXETER GROUPS AND "linearly correlated with Shannon entropy"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if (d * n) % 2 != 0 or d >= n:
            return None
        graph = {i: set() for i in range(n)}
        edges_added = 0
        while edges_added < d * n // 2:
            u, v = random.sample(range(n), 2)
            if u not in graph[v] and v not in graph[u]:
                graph[u].add(v)
                graph[v].add(u)
                edges_added += 1
        return graph

    def is_connected(graph):
        visited = set()
        stack = [0]
        while stack:
            node = stack.pop()
            if node not in visited:
                visited.add(node)
                stack.extend(neighbor for neighbor in graph[node] if neighbor not in visited)
        return len(visited) == len(graph)

    def compute_entropy(graph):
        n = len(graph)
        degree_sum = sum(len(neighbors) for neighbors in graph.values())
```

```

p = Fraction(degree_sum, 2 * n)
entropy = -p * math.log(p, 2) - (1 - p) * math.log(1 - p, 2)
return entropy

def compute_min_reflections(graph):
    if not is_connected(graph):
        return float('inf')

    n = len(graph)
    reflections = set()

    for node in range(n):
        neighbors = graph[node]
        for neighbor in neighbors:
            if neighbor < node:
                continue
            reflection = (node, neighbor)
            reflections.add(reflection)

    return len(reflections)

def is_valid_graph(graph):
    n = len(graph)
    degree_sum = sum(len(neighbors) for neighbors in graph.values())
    return degree_sum == 2 * n

metric_values = []
instances_tested = 0
n_max = 0
conjecture_holds = True
counterexample = ""

for n in [5, 10, 15, 20, 30, 40]:
    for _ in range(5):
        d = random.randint(2, min(n - 1, 4))
        graph = generate_d_regular_graph(n, d)
        if not is_valid_graph(graph):
            continue
        instances_tested += 1
        n_max = max(n_max, n)
        entropy = compute_entropy(graph)
        reflections = compute_min_reflections(graph)
        metric_values.append(reflections / entropy)

mean_value = sum(metric_values) / len(metric_values) if metric_values else float('nan')
std_dev = (sum((x - mean_value) ** 2 for x in metric_values) / len(metric_values)) ** 0.5 if n

return {
    "metric_name": "Reflections per Entropy",
    "metric_value": mean_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if not math.isnan(r["metric_value"])) / len(results)
    std_dev = (sum((r["metric_value"] - mean_value) ** 2 for r in results if not math.isnan(r["metric_value"]))) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(not math.isnan(r["metric_value"]) and r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif any(not math.isnan(r["metric_value"]) and not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not math.isnan(result["metric_value"]) and not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='not supported' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE no valid data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8101dff5.py", line 109, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8101dff5.py", line 83, in run_trial
    if not is_valid_graph(graph):
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8101dff5.py", line 69, in is_valid_graph
    n = len(graph)
        ~~~~~
TypeError: object of type 'NoneType' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the correlation coefficient and verifying the conjecture's support conditions. | next: Investigate

the cause of the crash in the test code and attempt to run the test again with proper error handling.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14427
2	preregistration	ollama_remote	glm4:latest	0	0	9938
3	novelty	ollama_remote	glm4:latest	0	0	8500
4	novelty	ollama_remote	glm4:latest	0	0	17746
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14369
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16546
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11294
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19132
9	judge	ollama_remote	glm4:latest	0	0	17231

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 129183 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/7a849ca4e68c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/7a849ca4e68c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/7a849ca4e68c.tar.gz (if generated)